

Desafio Integrador AgroSearch

DUPLA: Maria Clara Gouveia, Gabriel Guberson

1. Introdução

A necessidade de otimização no acesso às informações técnicas da startup *AgroTech Solutions* motivou o desenvolvimento do **AgroSearch**, um protótipo de motor de busca textual. O objetivo do projeto foi implementar o fluxo completo de Recuperação de Informação, desde o processamento textual bruto até a matemática de ranqueamento, garantindo uma solução customizada sem dependência de bibliotecas prontas como o scikit-learn para a criação do modelo.

2. Metodologia e Pipeline de NLP

O sistema foi estruturado em três fases principais integradas a uma interface web em Streamlit:

1. **Pré-processamento (Fase 1):** Implementamos funções puras em Python. O texto passa por *Normalização* (caixa baixa e remoção de acentos via unicodedata), *Tokenização* (através de expressões regulares re identificando blocos alfanuméricos), remoção de *Stopwords* (utilizando um conjunto manual para evitar dependências externas complexas de servidor) e um algoritmo de *Stemming* customizado que avalia e corta sufixos comuns no português (ex: "-ção", "-ões", "-ando").
2. **Índice Invertido (Fase 2):** Criou-se um mapeamento em dicionário (HashMap) onde cada termo único extraído do corpus pré-processado aponta para os identificadores dos documentos que o contêm. Isso garante uma recuperação $O(1)$ na verificação de existência do termo.
3. **Cálculo TF-IDF (Fase 3):**
 - a. O *Term Frequency* (TF) foi calculado como a razão entre as ocorrências do termo no documento e o tamanho total do documento.
 - b. O *Inverse Document Frequency* (IDF) aplicou a fórmula do $\log_{10}(N / df)$, penalizando palavras muito comuns.

3. Desafio Bônus: Similaridade de Cosseno

Para lidar de forma eficaz com consultas compostas (múltiplas palavras), não nos restringimos à soma simples do TF-IDF. Construímos um algoritmo que mapeia um espaço vetorial com a dimensão do vocabulário do *corpus*. O vetor da Query e os vetores dos Documentos são alocados nesse espaço, e aplicamos a métrica angular da *Similaridade de Cosseno* ($\frac{A \cdot B}{\|A\| \cdot \|B\|}$). Isso melhora drasticamente o ranqueamento, abstraindo o tamanho do documento e focando na orientação semântica.

4. Resultados e Análise da Interface

Através do menu lateral da aplicação Streamlit, o usuário pode ligar e desligar o *Stemming* e as *Stopwords*. Observa-se que:

- **Sem Stemming:** A busca por "lagarta" não encontra o Documento 4 ("Lagartas desfolhadoras...").
- **Com Stemming:** O termo e o documento convergem para o radical "lagart", garantindo o match correto, comprovando o valor do processamento morfológico em motores de busca de campo.

5. Trabalho em Equipe e Divisão de Tarefas

O projeto foi desenvolvido colaborativamente por dois integrantes, com as responsabilidades divididas de forma estratégica para otimizar a entrega:

- **Gabriel:** Ficou responsável pela arquitetura de *Processamento de Linguagem Natural (NLP)* e *Front-end*. Desenvolveu a Fase 1 (pipeline de normalização, tokenização, stopwords e o stemmer iterativo) e construiu a interface iterativa utilizando a biblioteca *Streamlit*, integrando a lógica de exibição do DataFrame interativo e o layout da barra de pesquisas.
- **Maria Clara:** Encarregou-se da *Engenharia de Busca* e das *Métricas Matemáticas*. Construiu a lógica da Fase 2 (Índice Invertido na memória) e Fase 3 (Cálculos de TF e IDF). Além disso, idealizou e codificou a lógica matemática do Desafio Bônus (Similaridade de Cosseno vetorial) e o sistema de ranqueamento descendente dos resultados.

6. Conclusão

O desenvolvimento "do zero" consolidou profundamente a compreensão da matemática por trás de buscadores como o Elasticsearch. O **AgroSearch** atende aos requisitos iniciais, fornecendo respostas precisas em milissegundos, com uma interface que facilita o trabalho técnico diário.